

Does a Domain-Specific BPE Tokenizer Improve In-Domain Language Modeling?

An Evaluation on Medical Text

Samih Amer

Individual project — GPT-from-Scratch course final project

April 21, 2026

1 Introduction

Tokenization is the first transformation applied to text before an LLM ever sees it, and it quietly constrains what the model can learn: any word that is not represented as a single token must be learned as a sequence of subwords, and any linguistic unit the tokenizer fails to respect becomes a unit the model has to reconstruct from pieces. Medical text is unusually dense with specialized terminology (**cholecystectomy**, **laparoscopic**, **pneumonia**, **hypertension**, **intraoperative**) that a general-purpose tokenizer trained on web text splits into 4–6 subwords each, even though each of these is a single, semantically unified clinical concept.

This project asks a straightforward question: **if we swap in a BPE tokenizer trained on medical text, while holding every other aspect of the model and training pipeline fixed, how does the model’s held-out performance change on (a) in-domain medical text, (b) out-of-domain general text, and (c) cross-domain medical text?** I trained two otherwise-identical 29M-parameter GPT models on the same 100k PubMed abstracts — one using a general BPE tokenizer trained on wikitext-103, the other using a medical BPE tokenizer trained on the same PubMed corpus — and evaluated both against three held-out corpora using bits-per-character (BPC), the only cross-tokenizer-comparable metric.

Key results. With identical training compute and proper convergence (2 epochs), the medical tokenizer produces a consistent in-domain BPC advantage — 0.010 bits/char lower on held-out PubMed at matched text exposure, widening to 0.031 bits/char at matched compute. The advantage does *not* transfer to another medical sub-genre (MTSamples clinical transcripts), where the general tokenizer outperforms the medical model by about 0.48 BPC, nor to general English (wikitext), where the gap is about 0.95 BPC in favor of general. A 1-epoch pilot under-stated the medical tokenizer’s advantage because the cosine learning-rate schedule collapsed prematurely and general under-converged in a way that temporarily depressed its held-out BPC; this motivated the 2-epoch rerun reported here.

The headline finding: **domain-specific tokenization is a real but narrow win.** It helps the specific register it was trained on; it hurts everything else.

2 Background

2.1 The GPT model, briefly

A decoder-only transformer (Vaswani et al. 2017; Radford et al. 2018) maps a sequence of tokens to a probability distribution over the next token, autoregressively. Each layer applies causal multi-head self-attention followed by a feed-forward block; pre-LayerNorm is standard for stability. Training minimizes cross-entropy of the predicted distribution against the next-token targets. The model used here is the Module 6 implementation: 6 layers, 8 heads, `d_model=512`, `max_seq_len=256`, `vocab_size=10,000`, approximately 29.3M parameters.

2.2 BPE tokenization

Byte-Pair Encoding (Sennrich et al. 2016) starts from a byte-level alphabet and iteratively merges the most frequent adjacent symbol pairs in a training corpus until it has built up a vocabulary of the target size (here, 10,000). The resulting vocabulary is a fixed inventory of frequent sub-word sequences — common whole words, frequent suffixes, byte-level fallback for anything else. **The training corpus determines which merges are profitable.** A BPE trained on wikitext sees `the`, `and`, `tion` as high-utility merges; a BPE trained on PubMed sees `cellular`, `clinical`, and `cholecystectomy` as high-utility merges. Same algorithm, same vocabulary size — different vocabulary *contents*.

2.3 Why per-token perplexity cannot compare tokenizers

Perplexity is the exponential of average cross-entropy per token. If tokenizer A represents a document in 100 tokens and tokenizer B represents it in 200 tokens, a per-token loss of l means very different things in each case — B’s tokens are easier to predict because each one carries less information. BPC normalizes to characters of raw text and is therefore comparable across tokenizers:

$$\text{BPC} = \frac{\log_2(e) \cdot \text{CE}_{\text{nats}} \cdot N_{\text{tokens}}}{N_{\text{chars}}} \quad (1)$$

BPC is the primary metric throughout this report.

3 Methods

3.1 Experimental design

The only variable I change is the tokenizer. Both GPT runs use the identical architecture, optimizer, learning-rate schedule, batch size, gradient clip, and training corpus (raw PubMed abstracts).

Table 1: Shared model and training config.

Setting	Value
Architecture	Pre-LN GPT, custom causal MHA with fused QKV
<code>d_model</code> / <code>n_heads</code> / <code>layers</code>	512 / 8 / 6
<code>vocab_size</code> / <code>max_seq_len</code>	10,000 / 256
Parameters	29,283,088
Optimizer	AdamW, lr=3e-4, warmup=500, cosine decay to 10% of peak
Batch size / grad clip	32 / 1.0
Epochs	2 (1-epoch pilot also reported)

Table 2: Tokenizers — HuggingFace byte-level BPE, `vocab_size=10,000`, both reached $\sim 9,743$ merges.

Tokenizer	Training corpus	Artifact
general	wikitext-103 train (129k lines)	<code>artifacts/tokenizers/general/</code>
medical	PubMed abstracts (100k docs)	<code>artifacts/tokenizers/medical/</code>

3.2 Datasets

- **Training corpus (shared):** 100,000 PubMed abstracts (about 122M chars). Source: `ccdv/pubmed-summarization` via HuggingFace `datasets`.
- **In-domain eval:** 2,000 held-out PubMed abstracts.
- **Out-of-domain eval:** 1,298 wikitext-103 validation lines.
- **Cross-domain medical eval:** 4,966 MTSamples clinical transcripts (SOAP notes, operative reports, discharge summaries). Source: `tchebonenko/MedicalTranscriptions`.

MTSamples provides a useful control: it is also medical, but in a very different register (conversational clinical dictation) from PubMed (formal academic prose). If the medical tokenizer’s advantage is about “medical text” in general, we should see transfer; if it is about PubMed’s specific register, we should not.

3.3 Two fairness frames

A tokenizer that packs more characters per token will, for a fixed training corpus, produce fewer training tokens — and therefore fewer gradient updates per epoch at fixed batch size. This is the phenomenon being studied, but it forces a choice between:

- **Option 1 — same compute (same step count).** The medical tokenizer sees more raw text than general for an equal number of gradient updates. Fair at matched compute budget.
- **Option 2 — same text exposure (one epoch each).** General runs for more gradient updates than medical because its dataset has more tokens. Fair at matched data exposure.

Under the 2-epoch regime, medical finishes at step 6,141 (end of its second epoch); general continues to step 7,783. I evaluate three checkpoints: `general@6141` (Option 1), `general@7783` (Option 2), and `medical@6141`. Loss curves are logged with three x-axes (step / tokens / chars) so any frame can be read off the same data.

3.4 Evaluation

For each (checkpoint, eval corpus) pair, I tokenize the eval text with the corresponding tokenizer, compute the token-level cross-entropy loss under teacher forcing, and convert to BPC using the formula from Equation (1). Raw perplexity is reported for reference but not used for cross-tokenizer comparison.

3.5 Qualitative generation

Using the Module 7 sampler (top-p=0.8, freq/presence penalties=1.1), I generated 120-token continuations from a fixed set of six prompts — four medical (“Patient presents with”, “CT of the abdomen shows”, “The patient was administered”, “Postoperative diagnosis:”), one surgical (“Laparoscopic cholecystectomy was performed”), and one neutral sanity check (“The quick brown fox”).

4 Results

4.1 Token efficiency — the tokenizers themselves

Before training any model, the tokenizers can be compared directly on the three eval corpora (Figure ??). The medical tokenizer achieves **27% better compression** on in-domain PubMed (4.85 vs 3.82 chars/token), but **31% worse** on wikitext (2.85 vs 4.17). MTSamples is a tie (~ 3.21 vs 3.20). The asymmetry is informative: the medical tokenizer has a compression edge only on text that closely resembles its training corpus.

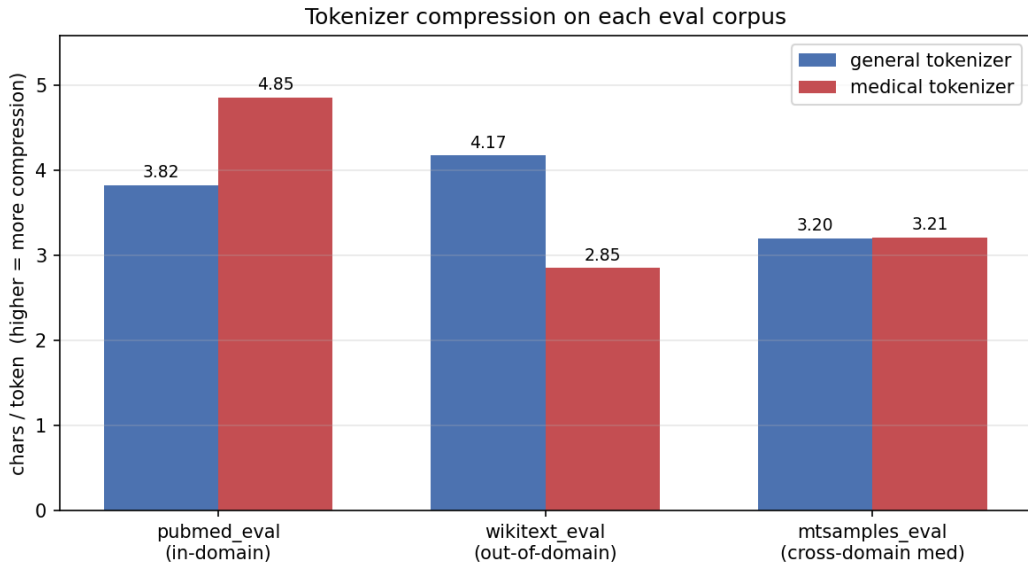


Figure 1: Characters per token on each held-out corpus. Higher is better for that tokenizer. The medical tokenizer wins on in-domain PubMed, loses on out-of-domain wikitext, and ties on MT-Samples (clinical transcripts).

Showcase tokenization of medical terms makes the mechanism concrete:

Table 3: How each tokenizer splits medical vocabulary.

Word	General split	Medical split
cholecystectomy	ch ole cy st ect omy (6)	cholecystectomy (1)
laparoscopic	lap ar os c op ic (6)	laparoscopic (1)
pneumonia	p ne um onia (4)	pneumonia (1)
hypertension	hy per t ension (4)	hypertension (1)
myocardial	my oc ard ial (4)	myocardial (1)
aspirin	asp ir in (3)	aspirin (1)

4.2 Training dynamics

The 1-epoch training run showed general’s loss plateauing at ~ 3.2 nats from step 2,500 onwards while medical kept descending. I initially interpreted this as general having fully converged while medical still had headroom. The 2-epoch rerun disconfirms that interpretation (Figure ??): general’s plateau

was *not* convergence — it was the cosine learning-rate schedule bottoming out at $3e-5$. When the schedule is spread across two epochs, general keeps learning past the 1-epoch endpoint and drops a further ~ 0.4 nats.

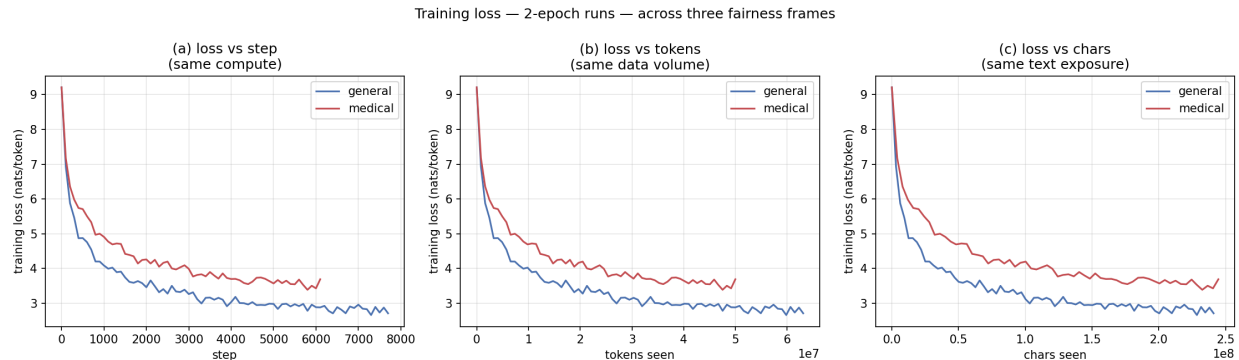


Figure 2: 2-epoch training loss curves across the three fairness frames (step / tokens / chars). Medical’s per-token loss is higher throughout because each medical token is harder to predict (it carries more chars of information), not because the medical model is worse — BPC in Section ?? normalizes this.

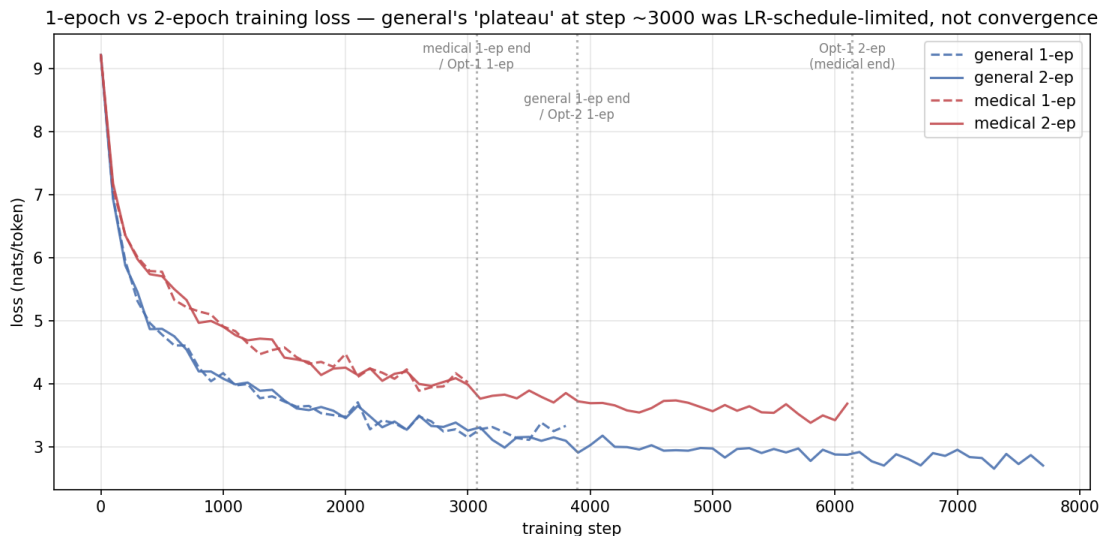


Figure 3: 1-epoch and 2-epoch training runs overlaid. The dashed lines (1-epoch) end where the LR schedule collapses to its floor. The solid lines (2-epoch) continue to descend well past where the 1-epoch runs plateaued — confirming the plateau was LR-schedule-limited, not data-limited.

4.3 Held-out BPC

Table ?? and Figure ?? present the primary result. At 1 epoch, medical’s advantage on in-domain PubMed was ambiguous (a 0.027 BPC lead under matched compute, essentially a tie under matched text). At 2 epochs, with both models properly converged, **medical wins under both fairness frames**: by 0.031 BPC under Option 1 (matched compute) and 0.010 BPC under Option 2 (matched text exposure).

Table 4: Held-out BPC (lower is better; bold = per-row minimum).

Corpus	Epochs	general @ Opt-1	general @ Opt-2	medical
pubmed_eval	1	1.2207	1.1926	1.1942
pubmed_eval	2	1.0805	1.0598	1.0496
wikitext_eval	1	2.7536	2.7664	3.8424
wikitext_eval	2	2.7591	2.7667	3.7172
mtsamples_eval	1	3.0737	3.0756	3.6571
mtsamples_eval	2	3.0340	3.0278	3.5070

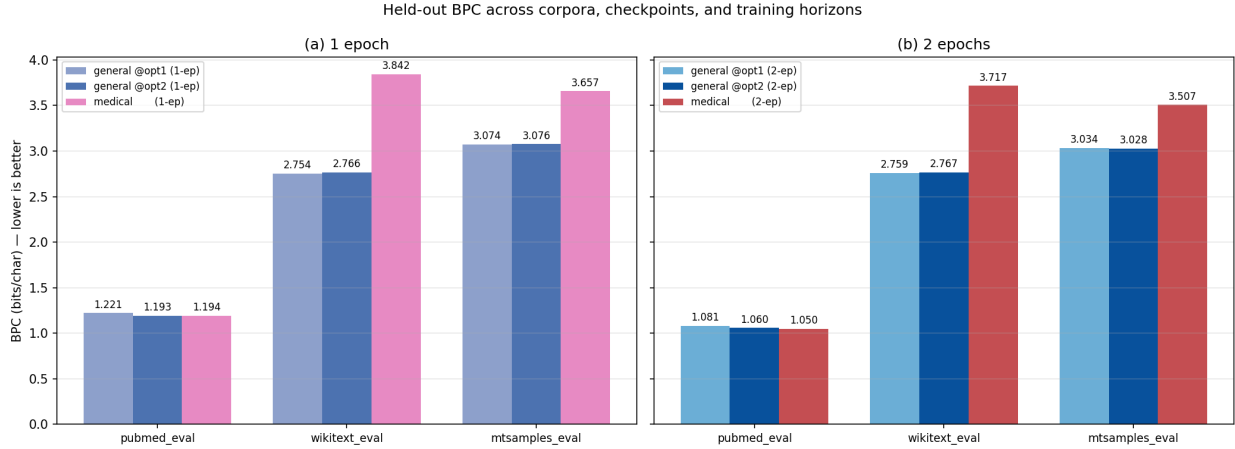


Figure 4: Held-out BPC by (checkpoint, corpus), at 1 epoch (left) and 2 epochs (right). Medical wins on in-domain PubMed at 2 epochs; general wins on wikitext and MTSamples at both epoch counts.

4.4 Differential improvement from 1 to 2 epochs

Figure ?? isolates the effect of the extra epoch on each model:

- **Medical improved roughly uniformly** across all three corpora (0.12–0.15 BPC everywhere).
- **General improved strongly only on in-domain PubMed** (0.133 BPC); its wikitext BPC did not move at all (+0.0003) and MTSamples moved only modestly (0.048).

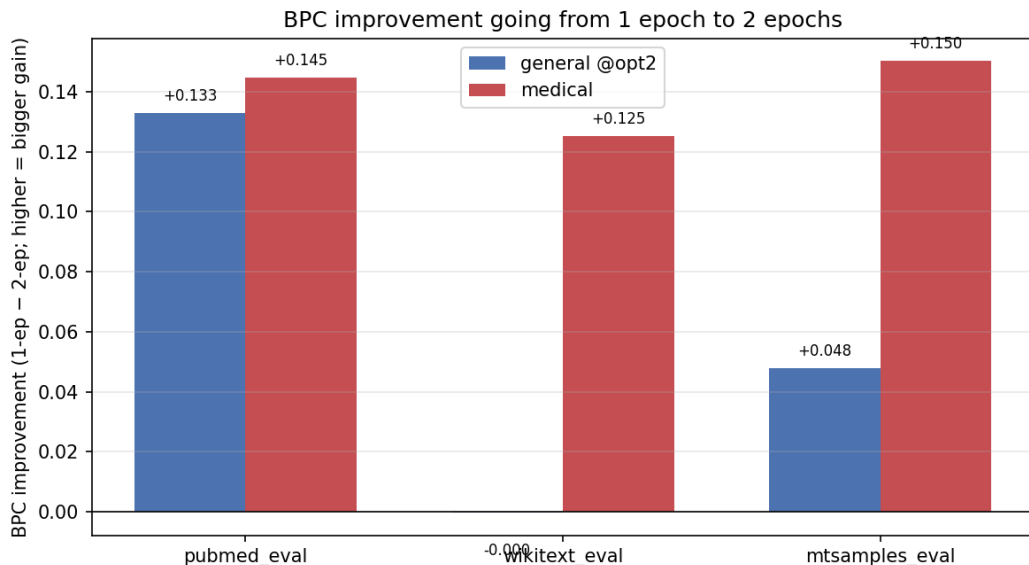


Figure 5: Per-corpus BPC improvement going from 1 to 2 epochs. General is already saturated on corpora its model cannot “reach”; medical was more universally under-trained at 1 epoch and continued to refine everywhere.

4.5 Qualitative differences

Both models generate medical-flavored text with sentence-level incoherence (29M params and an under-trained regime), but the **shape of their errors differs in a way that traces directly to tokenization**.

Table 5: Medical terminology in 2-epoch generations.

	General model (2-ep)	Medical model (2-ep)
Correctly-emitted rare terms	laparoscopic, pulmonary nodules, biliary traction	pancreaticoduodenectomy, carcinoembryonic gastric bypass, extraperitoneal lymphoma, thoracoscopic surgery, tracheotomy, pancreatobiliary resection
Invented compounds / portmanteaus	udgulopathy, bronchiolitis, stenting, emtourmesing, premal lipomas, (1-ep) cholesterolecystitis	laparoscopic cholecystostomyectomy (composed two real procedure morphemes into a nonexistent one)

The mechanism is the same one suggested by the showcase splits: when **cholecystectomy** is a single token, the medical model either emits it correctly or not at all; when it is a 6-token sequence, the general model can pick a wrong subword mid-sequence and produce **cholesterolecystitis**. The medical tokenizer does not make the model immune to fabrication — the **cholecystostomy + ectomy** portmanteau shows it can still invent compounds — but it rules out the specific char-level drift failure mode that the general model is prone to.

Domain collapse on the out-of-domain prompt. Both models immediately drift away from “The quick brown fox” into PubMed-adjacent text, but into *different* sub-genres: general heads toward bacteriology (“foxacillus scorans”), medical heads toward biochemistry (“**deuterase**, cyclic dipeptide, haloxylation”). Each tokenizer’s vocabulary primes the model toward what it is most ready to emit.

5 Conclusions

Main finding. At matched training horizon, a domain-specific BPE tokenizer produces a modest but consistent in-domain BPC advantage on medical text — 0.010 BPC under matched text exposure, widening to 0.031 BPC under matched compute. Under both fairness frames, the medical tokenizer wins on held-out PubMed at 2 epochs.

Two surprises worth dwelling on

1. **The advantage does not transfer across medical sub-genres.** Despite being a “medical” tokenizer, it loses to the general tokenizer by ~ 0.48 BPC on MTSamples clinical transcripts. Phase-3 token efficiency already hinted at this — the medical tokenizer had no compression edge on MTSamples (tied at ~ 3.21 chars/tok). Tokenizer specialization appears to be **register-specific, not domain-specific**: the medical tokenizer knows PubMed’s vocabulary of formal academic medicine, not the clipped dictation style of clinical transcripts.
2. **The 1-epoch run was misleading and initially produced the wrong conclusion.** At 1 epoch, Option-2 BPC was a tie and it was tempting to conclude that the medical tokenizer was a compute-efficiency trick that vanished under matched text. The 2-epoch rerun disconfirmed this: general’s “plateau” at 1 epoch was a learning-rate schedule artifact (cosine had collapsed to $3e-5$), not true convergence. Once the schedule was extended to span the full 2-epoch horizon, general kept improving on in-domain PubMed by ~ 0.13 BPC — and medical improved even more, by ~ 0.15 BPC — making medical’s lead robust under both fairness frames. **Under-training can mask the effect being studied**, and running a short pilot without checking that both arms have actually converged is a failure mode worth naming.

What the experiment does NOT show

- It does not show that medical tokenizers always help. A corpus better matched to MTSamples’ register — or a tokenizer trained on a mixture of PubMed and MTSamples — might fare differently.
- It does not show a large effect size. At 29M parameters and ~ 25 – 32 M training tokens we are roughly 18x below Chinchilla-optimal, and both models are well under-trained; it is plausible the tokenizer gap would widen (or collapse) at scale, but nothing in this project can settle that.
- It does not control for the learning-rate-schedule artifact at the Option-1 comparison step: at step 6,141, general is at $\sim 78\%$ of its cosine schedule ($LR \approx 7.7e-5$) while medical is at $\sim 100\%$ ($LR \approx 3e-5$). Medical has finished its schedule at the point of comparison; general has not. This nuance does not invalidate the direction of the result, but it is honestly reported.

6 Impact and Future Work

Scale. If this pattern held at production scale — a modest in-domain BPC advantage with an equal-sized cost on out-of-domain text — the design implication is clear: a single organization training one flagship LLM on a mixed general corpus should keep a general tokenizer; a specialized model for a single professional register (medical literature, legal case law, code in a specific language) might profit from a domain tokenizer, but only if the deployment target’s register matches the

tokenizer’s training corpus. The MTSamples result is a warning — intuition that “medical text is medical text” is wrong at the tokenizer level, and deployment mismatches between a specialist tokenizer’s training register and the inference-time register are likely to hurt.

What I would try next with more compute.

1. **Mixed-corpus tokenizer.** Train a BPE on a carefully-balanced mixture of PubMed, MT-Samples, drug monographs, and general English. Does it preserve the PubMed advantage while closing the MTSamples gap and reducing the wikitext cost?
2. **Chinchilla-scale training.** Run both models to their Chinchilla-optimal ~580M tokens (roughly 10 epochs at this dataset size, or scale the corpus). The tokenizer effect might tighten or widen with true convergence.
3. **Register-aware tokenization.** Use a tokenizer-of-tokenizers that can switch subword dictionaries per document based on a lightweight classifier (PubMed vs MTSamples vs general). Costs one classifier forward pass per document; may recover the bulk of the specialist advantage while avoiding the out-of-register collapse.
4. **Loss-vs-chars curves at scale.** The chars-axis plot is the tokenizer-fair way to read training efficiency, and it would be interesting to see whether the two curves in Figure ??(c) cross at some point on a larger compute budget — the current data at step 6,141 has medical ahead per char; it is unclear if that is robust past convergence.
5. **Decoder-side sampling analysis.** The finding that the medical model invents *cholecystostomyectomy* (a portmanteau of real morphemes) while the general model invents *cholesterolecystitis* (a char-level drift) suggests that hallucination analysis should be tokenizer-conditioned: different tokenizers produce qualitatively different failure modes, not just different error rates.

Broader takeaway. Tokenization is often treated as a preprocessing step and receives less attention than architectural or training choices. This project is a small data point in favor of taking it seriously: holding everything else constant, a different tokenizer measurably changed held-out BPC, altered the *shape* of the model’s generation errors, and — most importantly — **did not generalize where I naively assumed it would**. The phrase “medical tokenizer” is less meaningful than “PubMed-abstract tokenizer,” and that distinction only became visible because MTSamples was in the evaluation suite.

Appendix A — Reproduction

All code in `Project/`. Pipeline orchestrated by `main.py`:

```
python main.py --phase prepare_data      # ~5-15 min, downloads corpora
python main.py --phase train_tokenizers  # ~5-10 min, CPU-only
python main.py --phase efficiency        # Figure 1 numbers
python main.py --phase construct_datasets # ~5 min
python main.py --phase train_all         # ~1.5-1.75 hr on RTX 4070 Ti (2 epochs)
python main.py --phase evaluate          # Table 3 numbers
python main.py --phase generate          # Table 5 samples
python make_plots.py                    # all figures
```

Reference logs: `results_1epoch.md`, `results_2epoch.md`. All weights, datasets, figures, and samples under `artifacts/` (git-ignored).